

Automata Theory Questions from 38

Anik Chand

June 2026

Question 38

Q38. Explain the Halting Problem and prove that it is undecidable. Define decidable and undecidable problems. Explain with examples.

Halting Problem

The Halting Problem asks:

“Given a Turing Machine M and an input string w , determine whether M eventually halts when executed on w .”

Formally,

$$HALT = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$$

Proof of Undecidability

Assume that there exists a Turing Machine H that can solve the Halting Problem.

$$H(\langle M, w \rangle) = \begin{cases} YES, & \text{if } M \text{ halts on } w \\ NO, & \text{otherwise} \end{cases}$$

Construct another machine D as follows:

1. Take the description of a machine M as input.
2. Run $H(\langle M, M \rangle)$.
3. If H says “halts”, then loop forever.
4. If H says “does not halt”, then halt.

Now consider $D(D)$.

- If $D(D)$ halts, then by definition it must loop forever.
- If $D(D)$ loops forever, then by definition it must halt.

Both cases lead to a contradiction.
Therefore, our assumption that H exists is false.
Hence, the Halting Problem is **undecidable**.

The Halting Problem is undecidable.

Decidable Problems

A problem is called **decidable** if there exists a Turing Machine that always halts and gives the correct answer for every input.

Examples:

- DFA Acceptance Problem
- DFA Emptiness Problem
- Membership testing in regular languages

Undecidable Problems

A problem is called **undecidable** if no Turing Machine can always halt and correctly decide the answer for every input.

Examples:

- Halting Problem
- Post Correspondence Problem (PCP)
- Acceptance Problem of Turing Machines

Question 39

Q39. Compare Turing Machine with DFA and PDA. Why is TM more powerful?

Comparison of DFA, PDA and TM

Feature	DFA	PDA	TM
Memory	Finite States	Stack	Infinite Tape
Language Class	Regular	Context-Free	Recursively Enumerable
Read Input	Yes	Yes	Yes
Write on Tape	No	No	Yes
Move Left/Right	No	No	Yes
Storage Capacity	Fixed	Unlimited Stack	Unlimited Tape
Power	Least	More than DFA	Most Powerful

Why is Turing Machine More Powerful?

A Turing Machine has an infinite tape that acts as memory. It can:

- Read symbols.
- Write symbols.
- Move the tape head left or right.
- Modify previously written information.

A DFA has no external memory, while a PDA has only one stack. Therefore,

$$\text{DFA} \subset \text{PDA} \subset \text{TM}$$

A Turing Machine can simulate both DFA and PDA, but the reverse is not possible. Hence,

Turing Machine is more powerful than DFA and PDA.

Question 40

Q40. Define Turing Machine with its components. Prove that $L = \{ww^R \mid w \in \{a,b\}^*\}$ cannot be accepted by any DPDA but can be accepted by an NPDA.

Definition of Turing Machine

A Turing Machine is a mathematical model of computation consisting of a finite control, an infinite tape, and a read/write head.

Formally,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where

- Q = finite set of states
- Σ = input alphabet
- Γ = tape alphabet
- δ = transition function
- q_0 = initial state
- B = blank symbol
- F = set of final states

Components of a Turing Machine

1. Infinite Tape
2. Read/Write Head
3. Finite Control
4. Input Alphabet
5. Tape Alphabet
6. Transition Function

Language

$$L = \{ww^R \mid w \in \{a,b\}^*\}$$

Examples:

$$\epsilon, aa, bb, abba, baab, abaaba, \dots$$

This language contains all even-length palindromes.

Why DPDA Cannot Accept It

A DPDA must make deterministic decisions.

For a string in L , the automaton must determine the exact middle point where the first half ends and the second half begins.

For example,

abba

The machine must know after reading “ab” that it should stop pushing and start popping.

A DPDA cannot deterministically identify this midpoint without additional information.

Therefore no DPDA can recognize

$$L = \{ww^R\}.$$

Why NPDA Can Accept It

An NPDA can nondeterministically guess the midpoint.

1. Push symbols while reading the first half.
2. Nondeterministically guess the middle.
3. Start popping and compare symbols.
4. Accept if all symbols match and the stack becomes empty.

Hence an NPDA can recognize all strings of the form

$$ww^R.$$

Therefore,

$L = \{ww^R\}$ is accepted by NPDA but not by DPDA.

Question 45

Q45. Define Context-Free Grammar (CFG). Explain its components with an example.

Definition

A Context-Free Grammar (CFG) is a formal grammar used to generate context-free languages.

A CFG is represented by:

$$G = (V, T, P, S)$$

where

- V = Set of variables (non-terminals)
- T = Set of terminals
- P = Set of productions
- S = Start symbol

Example

Consider

$$G = (\{S\}, \{a, b\}, P, S)$$

with productions

$$S \rightarrow aSb \mid \epsilon$$

This grammar generates

$$L = \{a^n b^n \mid n \geq 0\}$$

Sample strings:

$$\epsilon, ab, aabb, aaabbb, \dots$$

Components

1. **Non-terminals:** $\{S\}$
2. **Terminals:** $\{a, b\}$
3. **Productions:** $S \rightarrow aSb \mid \epsilon$
4. **Start Symbol:** S

CFGs are used to describe context-free languages.

Question 46

Q46. Explain the difference between Left Linear Grammar and Right Linear Grammar with examples.

Left Linear Grammar

A grammar is left linear if the non-terminal appears at the left end of the production.
General form:

$$A \rightarrow Ba \quad \text{or} \quad A \rightarrow a$$

Example

$$S \rightarrow Aa$$

$$A \rightarrow Ab \mid b$$

Right Linear Grammar

A grammar is right linear if the non-terminal appears at the right end of the production.
General form:

$$A \rightarrow aB \quad \text{or} \quad A \rightarrow a$$

Example

$$S \rightarrow aA$$

$$A \rightarrow bA \mid b$$

Differences

Feature	Left Linear	Right Linear
Position of Variable	Left Side	Right Side
Production Form	$A \rightarrow Ba$	$A \rightarrow aB$
Generation Direction	Right to Left	Left to Right
Language Generated	Regular	Regular

Both generate regular languages.

Question 47

Q47. Prove that $L = \{a^{i^2} \mid i \geq 1\}$ is not regular.

Proof using Pumping Lemma

Assume that L is regular.

Then there exists a pumping length p .

Choose

$$w = a^{p^2}$$

Clearly,

$$|w| = p^2 \geq p$$

According to the Pumping Lemma,

$$w = xyz$$

such that

$$|xy| \leq p, \quad |y| > 0$$

Since the first p symbols are all a 's,

$$y = a^k, \quad k > 0$$

Pump y once:

$$xy^2z = a^{p^2+k}$$

Now

$$p^2 < p^2 + k < (p+1)^2$$

for sufficiently small k .

Therefore

$$p^2 + k$$

is not a perfect square.

Hence

$$xy^2z \notin L$$

which contradicts the Pumping Lemma.

Therefore

$$L = \{a^{i^2} \mid i \geq 1\} \text{ is not regular.}$$

Question 48

Q48. Define Regular Language and Regular Grammar. Give one example of each.

Regular Language

A language is called regular if it can be accepted by a DFA, NFA, or represented by a regular expression.

Example

$$L = (ab)^*$$

Strings:

$$\{\epsilon, ab, abab, ababab, \dots\}$$

Regular Grammar

A grammar is called regular if every production is either right-linear or left-linear.

General form:

$$A \rightarrow aB$$

or

$$A \rightarrow a$$

Example

$$S \rightarrow aA$$

$$A \rightarrow bA \mid b$$

This grammar generates

$$L = \{ab^n \mid n \geq 1\}$$

Every regular grammar generates a regular language.

Question 49

Q49. What are Closure Properties of Regular Languages? Explain closure under Union, Concatenation and Kleene Star.

Closure Property

A class of languages is said to be closed under an operation if performing that operation on languages of the class produces another language of the same class.

Regular languages are closed under many operations.

(a) Union

If

$$L_1 \text{ and } L_2$$

are regular languages, then

$$L_1 \cup L_2$$

is also regular.

Example

$$L_1 = \{a\}$$

$$L_2 = \{b\}$$

Then

$$L_1 \cup L_2 = \{a, b\}$$

which is regular.

(b) Concatenation

If

$$L_1 \text{ and } L_2$$

are regular, then

$$L_1 L_2$$

is also regular.

Example

$$L_1 = \{a\}$$

$$L_2 = \{b\}$$

Then

$$L_1 L_2 = \{ab\}$$

which is regular.

(c) Kleene Star

If L is regular, then

$$L^*$$

is also regular.

Example

$$L = \{ab\}$$

Then

$$L^* = \{\epsilon, ab, abab, ababab, \dots\}$$

which is regular.

Conclusion

Regular languages are closed under Union, Concatenation and Kleene Star.

Question 55

Q55. Define DFA and NFA. Write any two differences between them.

DFA (Deterministic Finite Automaton)

A DFA is a finite automaton in which for every state and input symbol there exists exactly one transition.

Formally,

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

$$\delta : Q \times \Sigma \rightarrow Q$$

NFA (Non-deterministic Finite Automaton)

An NFA is a finite automaton in which a state may have zero, one, or many transitions for the same input symbol.

Formally,

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

Differences

Feature	DFA	NFA
Transitions	Exactly one	Zero, one or many
ϵ -moves	Not allowed	Allowed (in ϵ -NFA)

Every NFA has an equivalent DFA.

Question 56

Q56. Explain Mealy and Moore Machines with examples.

Mealy Machine

A Mealy Machine produces output based on the current state and current input.

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where

$$\lambda : Q \times \Sigma \rightarrow \Delta$$

Output is associated with transitions.

Moore Machine

A Moore Machine produces output based only on the current state.

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where

$$\lambda : Q \rightarrow \Delta$$

Output is associated with states.

Differences

Feature	Mealy	Moore
Output depends on	State and Input	State only
Output attached to	Transitions	States
States required	Fewer	More

Mealy machines usually require fewer states than Moore machines.

Question 57

Q57. Let $L_1 = \{a^n \mid n \geq 0\}$ and $L_2 = \{b^n \mid n \geq 0\}$. Find $L_1 \cup L_2$, L_1L_2 and determine whether the result is finite or infinite.

Given

$$L_1 = \{\epsilon, a, aa, aaa, \dots\}$$

$$L_2 = \{\epsilon, b, bb, bbb, \dots\}$$

(a) Union

$$L_1 \cup L_2 = \{\epsilon, a, aa, \dots, b, bb, bbb, \dots\}$$

This contains all strings consisting only of a 's or only of b 's.

(b) Concatenation

$$L_1L_2 = \{a^mb^n \mid m, n \geq 0\}$$

Examples:

$$\epsilon, a, b, ab, aab, abb, aaabbb, \dots$$

(c) Finite or Infinite

Both

$$L_1 \cup L_2$$

and

$$L_1L_2$$

contain infinitely many strings.

Both languages are infinite.

Question 58

Q58. Explain the difference between ϵ (empty string) and $\emptyset$ (empty set) with examples.

Empty String (ϵ)

The empty string is a string containing no symbols.

$$|\epsilon| = 0$$

Example:

$$\epsilon \in \{a, b\}^*$$

Empty Set ($\emptyset$)

The empty set contains no elements.

$$|\emptyset| = 0$$

Example:

$$L = \emptyset$$

means the language contains no strings at all.

Differences

Feature	ϵ	$\emptyset$
Type	String	Set
Contains symbols	No	Not applicable
Elements present	One string	No elements

$$\epsilon \neq \emptyset$$

Question 59

Q59. Let $\Sigma = \{a, b\}$. List all strings of length ≤ 2 .

Strings of Length 0

ϵ

Strings of Length 1

a, b

Strings of Length 2

aa, ab, ba, bb

Therefore,

$$\Sigma^{\leq 2} = \{\epsilon, a, b, aa, ab, ba, bb\}$$

Total number of strings:

$$1 + 2 + 4 = 7$$

$$\Sigma^{\leq 2} = \{\epsilon, a, b, aa, ab, ba, bb\}$$

Question 60

Q60. Describe Kleene Star (*) and Kleene Plus (+) with suitable examples.

Kleene Star

For a language L ,

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

It allows zero or more repetitions.

Example:

If

$$L = \{ab\}$$

then

$$L^* = \{\epsilon, ab, abab, ababab, \dots\}$$

Kleene Plus

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

It allows one or more repetitions.

Example:

$$L^+ = \{ab, abab, ababab, \dots\}$$

Difference

$$L^* = L^+ \cup \{\epsilon\}$$

Kleene Star includes ϵ , whereas Kleene Plus does not.

Question 61

Q61. Given $\Sigma = \{a, b\}$ define (a) Σ^* (b) Σ^+ (c) Language of all strings with even length.

(a) Σ^*

Set of all strings over Σ including ϵ .

$$\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$$

(b) Σ^+

Set of all non-empty strings over Σ .

$$\Sigma^+ = \Sigma^* - \{\epsilon\}$$

(c) **Even Length Language**

$$L = \{w \in \Sigma^* \mid |w| \text{ is even}\}$$

Examples:

$$\epsilon, aa, ab, ba, bb, aabb, abab, \dots$$

Regular Expression:

$$((a + b)(a + b))^*$$

$$L = \{w \mid |w| \text{ is even}\}$$

Question 62

Q62. Explain the concept of length of a string. What is the length of ϵ ? Give examples.

Definition

The length of a string is the number of symbols present in it.

It is denoted by

$$|w|$$

Examples

$$|a| = 1$$

$$|ab| = 2$$

$$|abba| = 4$$

$$|aaabb| = 5$$

Length of Empty String

The empty string contains no symbols.

Therefore,

$$|\epsilon| = 0$$

$$\boxed{|\epsilon| = 0}$$

The length function is always a non-negative integer.

Question 64

Q64. Prove that $L = \{a^n b^n \mid n \geq 0\}$ is not a regular language using Pumping Lemma.

Proof

Assume that L is regular.

Then by the Pumping Lemma, there exists a pumping length p such that every string $w \in L$ with

$$|w| \geq p$$

can be written as

$$w = xyz$$

satisfying

$$|xy| \leq p, \quad |y| > 0$$

and

$$xy^i z \in L \quad \forall i \geq 0.$$

Choose

$$w = a^p b^p.$$

Since $|xy| \leq p$, the substring y consists only of a 's.

Let

$$y = a^k, \quad k > 0.$$

Pump y with $i = 2$:

$$xy^2 z = a^{p+k} b^p.$$

Now the number of a 's and b 's are different.

Therefore,

$$xy^2 z \notin L.$$

This contradicts the Pumping Lemma.

Hence,

$$\boxed{L = \{a^n b^n \mid n \geq 0\} \text{ is not regular.}}$$

Question 65

Q65. Prove that $L = \{a^n b^n\}$ is not regular.

Proof

Using the Pumping Lemma.

Assume L is regular.

Let p be the pumping length and choose

$$w = a^p b^p.$$

Then

$$w = xyz$$

with

$$|xy| \leq p, \quad |y| > 0.$$

Since the first p symbols are all a 's,

$$y = a^k, \quad k > 0.$$

Pump down ($i = 0$):

$$xz = a^{p-k} b^p.$$

Now the number of a 's and b 's are unequal.

Thus

$$xz \notin L.$$

Contradiction.

Therefore,

$$\boxed{L = \{a^n b^n\} \text{ is not regular.}}$$

Question 66

Q66. Define Closure Properties of Regular Languages with example.

Definition

A family of languages is said to be **closed** under an operation if performing that operation on languages from the family produces another language in the same family.

Regular languages are closed under:

- Union
- Intersection
- Complement
- Concatenation
- Kleene Star
- Difference
- Reversal

Example

Let

$$L_1 = a^*$$

and

$$L_2 = b^*.$$

Both are regular.

Their union is

$$L_1 \cup L_2 = a^* \cup b^*.$$

This language is also regular.

Hence regular languages are closed under union.

Regular languages remain regular after closure operations.
--

Question 67

Q67. What is Pumping Lemma for Regular Languages?

Statement

If L is a regular language, then there exists a positive integer p (called pumping length) such that every string

$$w \in L$$

with

$$|w| \geq p$$

can be decomposed as

$$w = xyz$$

satisfying:

$$|xy| \leq p$$

$$|y| > 0$$

$$xy^iz \in L \quad \forall i \geq 0.$$

Importance

The Pumping Lemma is mainly used to prove that a language is **not regular**.

Idea

A DFA has a finite number of states. Therefore, while processing a sufficiently long string, some state must repeat. The repeated portion can be “pumped” any number of times.

Pumping Lemma is a necessary property of all regular languages.

Question 68

Q68. Explain Arden's Theorem.

Statement

Let

$$R = Q + RP$$

where

$$\epsilon \notin P.$$

Then the unique solution is

$$R = QP^*.$$

Similarly, if

$$R = Q + PR,$$

then

$$R = P^*Q.$$

Importance

Arden's Theorem is used to obtain a regular expression from a finite automaton.

Example

Given

$$R = aR + b$$

Applying Arden's Theorem,

$$R = ba^*.$$

Hence,

$$\boxed{R = QP^*}$$

is the solution of

$$R = Q + RP.$$

Question 69

Q69. Define Regular Expression with an example.

Definition

A Regular Expression (RE) is an algebraic notation used to describe regular languages.
The basic regular expressions are:

$$\emptyset$$

$$\epsilon$$

$$a \quad (a \in \Sigma)$$

Using the operations:

- Union (+)
- Concatenation
- Kleene Star (*)

more complex expressions can be formed.

Example

Consider

$$(a + b)^*$$

This denotes all strings over the alphabet

$$\{a, b\}.$$

Examples:

$$\epsilon, a, b, ab, ba, aa, bb, aba, \dots$$

Another example:

$$ab^*$$

represents

$$\{a, ab, abb, abbb, \dots\}.$$

Regular expressions are used to represent regular languages.
--

Question 72

Q72. What is the difference between Mealy and Moore Machine?

Mealy Machine

A Mealy Machine generates output based on the current state and current input.

$$\lambda : Q \times \Sigma \rightarrow \Delta$$

Output is associated with transitions.

Moore Machine

A Moore Machine generates output based only on the current state.

$$\lambda : Q \rightarrow \Delta$$

Output is associated with states.

Differences

Feature	Mealy Machine	Moore Machine
Output depends on	State and Input	State only
Output attached to	Transition	State
Response speed	Faster	Slower
Number of states	Usually fewer	Usually more

Mealy machines generally require fewer states than Moore machines.

Question 74

Q74. What is the difference between DFA and NFA?

Differences Between DFA and NFA

Feature	DFA	NFA
Transition Function	$\delta : Q \times \Sigma \rightarrow Q$	$\delta : Q \times \Sigma \rightarrow 2^Q$
Transitions for Input	Exactly one	Zero, one, or many
ϵ -Transition	Not allowed	Allowed in ϵ -NFA
Implementation	Simple	Compact
Acceptance	Single computation path	Any computation path

Both DFA and NFA recognize exactly the same class of regular languages.

Question 75

Q75. Define DFA and NFA with example.

DFA

A Deterministic Finite Automaton (DFA) is a finite automaton in which each state has exactly one transition for every input symbol.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Example

DFA accepting strings ending in 1:

$$Q = \{q_0, q_1\}$$

$$F = \{q_1\}$$

Transitions:

$$\delta(q_0, 1) = q_1$$

$$\delta(q_1, 1) = q_1$$

$$\delta(q_0, 0) = q_0$$

$$\delta(q_1, 0) = q_0$$

NFA

A Non-deterministic Finite Automaton (NFA) is a finite automaton where multiple transitions may exist for the same input.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Example

NFA accepting strings containing at least one a :

$$Q = \{q_0, q_1\}$$

$$F = \{q_1\}$$

Transitions:

$$\delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_0, b) = \{q_0\}$$

Question 76

Q76. Differentiate between Terminal and Non-Terminal Symbols.

Terminal Symbols

Terminal symbols are the actual symbols of the language that cannot be replaced further.

Example:

$$\{a, b, 0, 1\}$$

Non-Terminal Symbols

Non-terminal symbols are variables used during derivation and can be replaced using production rules.

Example:

$$\{S, A, B\}$$

Differences

Feature	Terminal	Non-Terminal
Can be replaced	No	Yes
Appears in final string	Yes	No
Representation	Lowercase symbols	Variables
Examples	$a, b, 0, 1$	S, A, B

Terminals form the final strings, while non-terminals help generate them.

Question 77

Q77. Define Regular Grammar with an example.

Definition

A Regular Grammar is a grammar in which every production is either right-linear or left-linear.

Right-linear form:

$$A \rightarrow aB$$

or

$$A \rightarrow a$$

Left-linear form:

$$A \rightarrow Ba$$

or

$$A \rightarrow a$$

Example

$$S \rightarrow aA$$

$$A \rightarrow bA \mid b$$

This grammar generates

$$L = \{ab^n \mid n \geq 1\}$$

Examples:

$$ab, abb, abbb, \dots$$

Regular grammars generate regular languages.
--

Question 78

Q78. What is a Grammar? Explain its components with an example.

Definition

A Grammar is a set of rules used to generate strings of a language.

Formally,

$$G = (V, T, P, S)$$

where

- V = Set of non-terminals
- T = Set of terminals
- P = Set of production rules
- S = Start symbol

Example

Consider the grammar

$$S \rightarrow aSb \mid \epsilon$$

Then

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow aSb, S \rightarrow \epsilon\}$$

$$S = \text{Start Symbol}$$

This grammar generates

$$L = \{a^n b^n \mid n \geq 0\}$$

Examples:

$$\epsilon, ab, aabb, aaabbb, \dots$$

Components of Grammar

1. Non-Terminals (V)
2. Terminals (T)
3. Production Rules (P)
4. Start Symbol (S)

A grammar is a formal mechanism for generating a language.

Question 82

Q82. Explain reverse of a string and language with suitable examples.

Reverse of a String

The reverse of a string is obtained by writing its symbols in the opposite order.

If

$$w = a_1a_2a_3 \cdots a_n$$

then its reverse is

$$w^R = a_na_{n-1} \cdots a_2a_1$$

Examples

$$(abcd)^R = dcba$$

$$(1010)^R = 0101$$

$$(aab)^R = baa$$

$$\epsilon^R = \epsilon$$

Reverse of a Language

For a language L , the reverse language is

$$L^R = \{w^R \mid w \in L\}$$

Example

Let

$$L = \{ab, aab, baa\}$$

Then

$$L^R = \{ba, baa, aab\}$$

Reversal changes the order of symbols from left to right.

Question 83

Q83. Differentiate between Recursive and Recursively Enumerable Languages.

Recursive Language

A language is called **Recursive (Decidable)** if there exists a Turing Machine that halts on every input and correctly decides whether the input belongs to the language.

Recursively Enumerable Language

A language is called **Recursively Enumerable (RE)** if there exists a Turing Machine that accepts all strings in the language.

For strings not in the language, the machine may reject or may run forever.

Differences

Feature	Recursive	Recursively Enumerable
TM Halts on Every Input	Yes	Not Necessary
Decision Possible	Yes	No
Also Called	Decidable	Turing Recognizable
Acceptance	Correctly Decides	Only Recognizes

Examples

- DFA Acceptance Language — Recursive
- Halting Problem Language — Recursively Enumerable but not Recursive

Every Recursive language is RE, but not every RE language is Recursive.

Question 85

Q85. Define Decidable and Undecidable Problems with examples.

Decidable Problems

A problem is said to be **Decidable** if there exists a Turing Machine that always halts and gives the correct answer for every input.

Examples

- DFA Acceptance Problem
- DFA Emptiness Problem
- Membership Testing in Regular Languages

Undecidable Problems

A problem is said to be **Undecidable** if no algorithm or Turing Machine can always halt and correctly decide the answer for every input.

Examples

- Halting Problem
- Post Correspondence Problem (PCP)
- Acceptance Problem of Turing Machines

Difference

Feature	Decidable	Undecidable
Algorithm Exists	Yes	No
Always Halts	Yes	Not Possible
Correct Decision	Yes	Not Guaranteed

Decidable problems can be solved algorithmically, while undecidable problems cannot.

Question 86

Q86. Define Turing Machine. Explain the model of Turing Machine.

Definition

A Turing Machine (TM) is a mathematical model of computation introduced by **Alan Turing** in 1936.

It consists of a finite control, an infinite tape, and a read/write head.

Formally,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where

- Q = Finite set of states
- Σ = Input alphabet
- Γ = Tape alphabet
- δ = Transition function
- q_0 = Initial state
- B = Blank symbol
- F = Set of final states

Model of Turing Machine

A Turing Machine consists of:

1. **Infinite Tape**
Used as memory and divided into cells.
2. **Read/Write Head**
Reads symbols from the tape and writes new symbols.
3. **Finite Control**
Controls the operation according to the transition function.
4. **Transition Function**
Determines:
 - Next state
 - Symbol to be written
 - Direction of movement (L or R)

Working of TM

For each move, the machine:

1. Reads the current symbol.
2. Writes a new symbol (if required).
3. Changes its state.
4. Moves the tape head Left or Right.

Diagram of TM

Finite Control $\longleftrightarrow$ Read/Write Head $\longleftrightarrow$ Infinite Tape

A Turing Machine is the most powerful standard model of computation.